

PPCensor: An Architecture for Real-time Pornography Detection in Video Streaming

Jackson Mallmann
Federal Institute Catarinense, jackson.mallmann@ifc.edu.br
Pontifical Catholic University of Paraná
(PUCPR), jackson.mallmann@pucpr.edu.br
Brazil

Eduardo Kugler Viegas
Pontifical Catholic University of
Paraná (PUCPR)
eduardo.viegas@pucpr.br
Brazil

Roger Robson dos Santos
Pontifical Catholic University of
Paraná (PUCPR)
robson.roger@ppgia.pucpr.br
Brazil

Altair Olivo Santin
Pontifical Catholic University of Paraná
(PUCPR)
altair.santin@pucpr.br
Brazil
Jhonatan Geremias
Pontifical Catholic University of
Paraná (PUCPR)
j.geremias@ppgia.pucpr.br
Brazil

ABSTRACT

Convolutional neural networks (CNN) models typically consist of several Gigabytes of data, hence, dedicated hardware and significant processing capabilities are required to perform these tasks. In addition, the detection task is typically performed offline because video frames are classified individually, and the video is categorized as inappropriate only after the evaluation of all the video frames. This paper proposes Private Parts Censor (PPCensor) with a CNN-based architecture for transparent and near real-time detection and obfuscation of pornographic video frame regions. Our contribution is two-fold. First, our work is the first approach that addresses pornographic content as an object recognition task. The goal is to perform real-time content filtering in a user-friendly manner, considering that an eventual false-positives will only obfuscate video frame regions, instead of blocking access to the entire video content. Secondly, PPCensor architecture is deployed in dedicated hardware, and detection is performed via a video-oriented streaming proxy. Consequently, video streaming content is assessed in real-time for all queried video URL. If pornographic video content is identified, it may block access or hide inappropriate areas in a remote website without user interaction or user device processing requirements. The evaluation results obtained based on over 50 thousand manually labeled objects have shown that PPCensor is able to detect private parts in videos using state-of-the-art CNN architectures while also achieving similar results to traditional image classification architectures. In addition, when deployed on a desktop device, PPCensor can handle up to 35 simultaneous real-time connections without additional processing.

KEYWORDS

Convolutional Neural Networks; Object-Oriented Private Parts Detection; Private Parts Object Dataset; Video-Oriented Streaming Proxy; Close Real-time Detection; Resource-Constrained Devices, Private Parts Censor;

1 INTRODUCTION

Over the last years, the dissemination of pornographic video over the Internet, including videos with explicitly pornographic content, has increased significantly [1]. For instance, in 2018, one of the most visited pornographic websites receives over 4 million daily video uploads [2]. Consequently, underaged users are prematurely and inappropriately exposed to pornography. The exposure to such undesirable content without parental consent may cause embarrassment or even psychological trauma [3]. Consequently, over 71% of underaged responsible are concerned with their infant web browsing activities [4].

In general, in the literature, convolutional neural networks (CNN) are utilized, and they have yielded promising results in the detection of pornography content in videos [5-9]. To achieve this goal, a video is split into a set of video frames, and each video frame is classified individually [10]. Hence, a single video can consist of several frames, considering that they are typically encoded in a frames/second ratio of 23.97 [11]. As a result, detection techniques cannot be executed in real-time. This is because these techniques assume that the entire set of video frames is available for detection. However, in real-life applications, videos are often viewed continuously in a streaming-based manner [12], for instance, an underaged person watching an online video on pornographic websites. Moreover, given that each video frame is classified individually, a single video may contain several pornographically labeled frames, even if the video does not have inappropriate content.

A CNN-based detection approach may misclassify some frames, and consequently, misclassify the video [13]. Therefore, video filtering via CNN-based techniques may present a high false-positive (FP) rate, a situation in which several normal videos are misclassified as pornographic or vice-versa, producing a false-negative (FN). To improve detection accuracy, traditional CNN-based detection techniques are often memory and CPU/GPU demanding [14]. As an example, a widely used CNN architecture may require up to 16 Gigabytes of RAM memory while processing only 4.26 frames/second in a high-end GPU [15]. Consequently, the CNN deployment to the end-user device becomes a challenging task, considering that available detection approaches in the literature usually do not consider the CNN deployment task, thus, ignoring the lack of available resources of the end-user device [16]. In addition, inappropriate videos are often viewed in resource-constrained devices such as smartphones, tablets among others. As a result, current

state-of-the-art CNN-based video filtering techniques are not suitable for deployment in such environments, considering that due to the high computational demands, these techniques often require costly and dedicated high-end hardware [13].

In light of this, this paper presents PPCensor, an architecture for real-time pornographic frame detection in video streaming. Our proposed technique is based on two main insights. First, to address real-time detection of pornographic frames in videos in a reliable manner, our work is the first approach to treat pornographic content as an object recognition problem. The goal is to hide only the inappropriate areas in a set of frames, instead of blocking access to the entire video. Consequently, the PPCensor is still reliable, given that it is possible to view the video, whereas potentially inappropriate frames are hidden. In addition, even if an FP occurs, the user will only experience a moment (some frames) with blurred (obfuscated) regions in the video, rather than incorrectly blocking access to the entire video. Therefore, the user experience when using the PPCensor is not significantly degraded owing to the occurrence of an FP. Second, to enable actual usage in production, PPCensor is implemented as a proxy server for video streaming. The main insight is to transparently assess pornography video occurrence, and classification of video streaming remotely. To achieve this objective, PPCensor is deployed in dedicated hardware, and it performs its detection by means of a proxy server. Consequently, it assesses the video content of all queried video URI (e.g., URL) in real-time. If pornographic video content is found, it may block access or hide inappropriate frames in the host by classifying the pornographic content without user intervention. Therefore, PPCensor only requires that the user utilize it as a proxy server, thus, enabling the filtering of inappropriate content regardless of the user device. In summary, this paper includes the following contributions:

- We provide a publicly available dataset with four annotated private parts of a human body (objects), namely, the *Private Parts Object Dataset* (PPO). The dataset is created based on the manual inspection of pornographic videos, resulting in 52,215 annotated objects. The PPO is the first publicly available dataset in the literature with annotated private parts;
- We present a novel pornography content detection technique that handles pornography detection as an object recognition problem. Our proposed approach, the first in the literature, can perform the detection of private parts with similar accuracy rates compared to traditional image-based state-of-the-art approaches;
- We present a novel CNN-based detection architecture (PPCensor) that is suited to the resource-constrained nature of the devices commonly used for viewing pornographic videos. The architecture implements the proposed object recognition process in a web proxy server. Therefore, it does not require device modifications, neither additional processing on the device, and executes remotely and transparently to the end-user;

The remainder of this paper is organized into sections. Section 2 presents the background with respect to CNN-based detection techniques and object recognition approaches. Section 3 describes related works. Section 4 presents a preliminary evaluation of state-of-the-art pornographic video detection techniques. Section 5 presents PPCensor. Our proposal is evaluated in Section 6. Finally, Section 7 summarizes the main conclusions of our work.

2 VIDEO CLASSIFICATION

In general, most video classification techniques split the video into frames [10], and then each frame is classified individually by applying an appropriate technique. To achieve this objective, a video classification technique is typically divided into three sequential modules; namely, *frame extractor*, *frame classification*, and *decision*. First, the *frame extractor* module is responsible for extracting the video frames that will be classified. The *frame classification* module then classifies each extracted video frame individually, for instance, as either normal or pornography. Finally, the *decision* module establishes the video class based on these video frames. In general, a video is classified based on the majority of its individual video frames.

2.1 Convolutional Neural Networks

Over the few last years, several detection techniques have been proposed for frame classification tasks. In such a context, CNN-based detection approaches have developed into the state-of-the-art procedure for image processing tasks [13]. A CNN typically consists of three-layer types, namely, convolutional, pooling, and fully connected [17] in which each architecture varies their disposition and the number of used layers.

The most prominent CNN for image processing includes the ResNet architectures [18] and Inception [19]. In both cases, it is necessary to initialize configuration parameters, such as the number of epochs, the learning rate, decay, and momentum [13]. To improve accuracy and reduce the training time, several investigators have relied on pre-trained CNN architectures, in a process known as transfer learning [20-21]. In this process, CNN uses the weights of a previously trained architecture, such as ImageNet training weights, a dataset with more than 1.2 million images divided into 1000 classes [22].

The classification procedure starts with an input layer, which receives the raw image as the input, i.e., a pixel matrix. The resulting data is then surpassed for each posterior layer until the output layer is reached. Finally, the output layer generates a probability vector for each considered class label as the output. For instance, the class probabilities for normal or pornographic frames are generated.

In the literature, a common approach for improving CNN accuracy is to increase its network complexity. For instance, several works have considered increasing the number of layers, thereby increasing the processing demands and memory size. To deploy a CNN-based detection technique, it is common to rely on dedicated hardware equipped with high-end GPUs. However, the deployment of such approaches is challenging for resource-constrained devices.

2.2 CNN-based Object Recognition

Traditional CNN-based video classification techniques classify a given input image as belonging or not belonging to a class (e.g. pornography or normal). However, in some applications, a given image may contain several regions of interest. An example of this is a system designed for person recognition in which a single input image may contain several persons for identification. In such a case, several image regions must be identified as opposed to the classification of the entire image. Consequently, CNN-based approaches have also been used for object recognition tasks in images [23]. The objective is to identify a bounding-box region (image region) that correctly includes the occurrence of the object in the image. Several approaches have been proposed to achieve this goal, which differs from feature extractors to the input image test size [24], such as the Faster R-CNN [25], Single Shot Detector (SSD) [26] and You Only Look Once (YOLO) [27].

Similar to classification techniques, object detectors are also training-based techniques. For instance, there are two main characteristics in Faster R-CNN [25]. First, the features are extracted from images and sent to the Region Proposal Network (RPN), which performs the feature extraction according to the test images. Second, the detection is performed, i.e., for each location, the probability of the existence of the searched object is calculated. The SSD architecture is executed in only one stage, given that they are considered to be fast and fitted for resource-constrained hardware [24] such as mobile phones and tablets. To improve accuracy, researchers often rely on more complex CNN architecture such as Faster R-CNN. Similar to image-based classification, it is possible to use transfer learning [20-21] to train object detectors. As such, these trained models facilitate the refinement of weights using pre-trained CNNs with transfer learning. In general, transfer learning in object recognition is achieved using the COCO dataset [28], which consists of 330 thousand images divided into 80 classes (objects).

In general, CNN-based object detectors are evaluated with respect to their detection throughput and Mean Average Precision (mAP). The mAP is computed according to the intersection over union (IoU). In general, in the literature, it is applied a threshold of 0.5 for each object bounding boxes. The IoU establishes the region of the identified bounding box that intersects with the target object region [23].

3 RELATED WORKS

The detection of pornographic content in videos has been widely studied in the literature. However, real-time detection is still in its infancy, particularly for approaches that are intended for deployment in the end-user device.

3.1 Pornography Content Detection

Over the last years, several approaches have been proposed for the detection of pornographic content. In general, the majority of these techniques can be divided into either skin-tone based or CNN-based [29].

For instance, a skin-tone based lightweight approach was proposed by the Brazilian Federal Police experts, namely, NuDetective [30]. The software uses the Ap-Apid algorithm [31] and automatically detects nudity in images according to the identified skin-tones. It was developed as a forensic tool to aid to retrieve pornographic files in seized computers. In contrast, F. Nian et al. [32] detected pornographic images using a CNN architecture trained based on transfer learning [20-21], and a training dataset consisting of 48,600 images (13,300 pornographic and 35,300 normal) obtained on the Internet. The authors relied on mid-level representations and adjustment of the training data as two key strategies. The main contribution of the authors is based on the application of the “fixed-point algorithm” [33], which enables the detection of different image sizes.

The detection of pornographic content in video is still in its infancy. In general, detection approaches are evaluated using publicly available datasets, such as Pornography-2k [34], Sensitive [9] and NPDI [35]. For instance, Moustafa et al. [5] applied several CNN architectures including AlexNet [22] and GoogleNet [36] for the detection of pornographic videos. In their work, a video was categorized as pornographic if the majority of its frames were classified as containing inappropriate content, as evaluated based on the NDPI [35] dataset.

Another approach was also proposed by Yahoo, namely, Yahoo Detector [6], which is a publicly available tool for finding *Not-Safe-For-Work* (NSFW) images. Their technique applies a ResNet50 [18] CNN architecture trained with transfer learning [20-21] based on ImageNet [22]. However, although these techniques yield reasonably accurate results, the used dataset is typically not shared. As such, their results cannot be compared. In Vitorino et al. [37], a CNN architecture (GoogleNet [36]) that was trained using transfer learning [20-21] was applied to the publicly available Pornography-2k dataset [34]. Using this dataset, previously proposed techniques could be evaluated, and a benchmark for accuracy comparison provided. For example, Yahoo Detector achieves only 88% accuracy when this dataset is used [37].

M. Perez et al. [7] also evaluated their technique using the Pornography-2k [34] and NPDI [35] datasets. The authors proposed the use of static and motion features for CNN-based classification of videos. Their approach significantly improved detection accuracy but cannot be implemented in real-time because require the entire video is needed for the extraction of motion-based features. In that investigation, the best method yielded a classification accuracy of 97.9 % for the NPDI [35] dataset. J. Wehrmann et al. [8] proposed a deep-learning architecture called ACORDE. Their detection architecture consisted of CNN and recurring networks for the detection of adult content in videos. Their experiments were performed using the NPDI [35] dataset and yielded reasonably accurate results, thereby reducing the number of FPs and FNs in relation to approaches that utilized this same dataset.

3.2 CNN-based Object Recognition

Several CNN architectures have been proposed for object recognition tasks that have been utilized in several fields. For instance, they have been used to recognize vehicles [38-40], people [41], , water surface objects [42], and even for crime scene evidence analysis [43].

Regarding object detection, X. Ou et al. [9] attempted to classify adult image/video by utilizing the deep multi-context network (DMCNet) [44], a hierarchical method that uses the feature output of a CNN model to train the RPN of the Faster R-CNN Object Detection. For comparison purposes, the authors generated the Sensitive dataset [9], and made comparisons with some datasets. However, although the authors applied an object recognition CNN architecture, they did not detect private parts.

Several approaches have been proposed for the detection of pornographic content in videos; however, their implementation in real-world applications remains an open challenge. This is because to increase accuracy, investigators typically extract features over several frames [5-6], or significantly increases the CNN complexity [7-9]. Nonetheless, real-time detection considering the actual deployment of their proposed technique remains unaddressed in the literature. However, a pornography detection approach based on object recognition techniques can significantly improve user experience. This is because FP only occurs in a blurred area of the video, instead of restricting access to the entire video. Thus, there are limited works that use object detection to locate private parts, as well as the detection of pornography.

4 THE CHALLENGE OF REAL-TIME DETECTION OF PORNOGRAPHIC VIDEOS

In this section, we evaluate the classification performance of publicly available detection approaches for pornographic content. Specifically, we first describe the used pornographic datasets, and then, we evaluate NuDetective [30] and Yahoo Detector [6] for both accuracy and throughput.

4.1 Datasets

In general, approaches for the detection of pornography are evaluated in a single dataset. Consequently, the accuracy obtained is expected to be evidenced when the approach is used in production. However, the use of a single dataset may introduce overfitting to detection techniques. This is because a video dataset is prone to several flaws, such as being specific to the environment in which it was acquired, the extracted video quality and encoding, the video length, in addition to other media characteristics.

As such, we evaluate pornography content detection using two datasets. Specifically, we consider the challenge of pornographic content detection in videos using the Pornography-2k [34] and UCF101 [45] datasets. The prior is widely used in the literature as a benchmark pornographic detection scheme. The dataset consists of 2 thousand videos in which 1 thousand are pornographic related and 1 thousand are normal videos. In contrast, the former consists of 13,320 normal videos. The goal is to obtain a realistic detection benchmark for both FP and FN rates. The FP rate denotes the ratio of normal videos that are misclassified as pornography, whereas the FN rate denotes the ratio of pornographic videos that are misclassified as normal. Both datasets include individuals from several ethnicities involved in different activities, and various video quality.

The dataset frames were extracted in a 10-frame interval. Therefore, one frame was extracted for further analysis at approximately every 0.3 seconds. In total, Pornography-2k dataset consisted of 1,376,035 frames, in which 389,808 frames are normal ones and 986,233 frames are pornographic content, whereas UCF101 [45] dataset consisted of videos with 240,810 normal frames.

4.2 The Reliability of Pornography Detection Schemes

Two pornography detection approaches, Yahoo Detector [6] and NuDetective [30], were evaluated in the assembled datasets. The evaluated techniques were executed on a desktop computer equipped with an Intel I7 CPU (quad-core), 16 GB of memory and a Nvidia Titan-XP GPU.

Table 1 shows the results for Yahoo Detector and NuDetective with respect to detection throughput and accuracy. It can be observed that NuDetective significantly outperforms Yahoo Detector in terms of throughput, reaching up to 411 frames/second, in contrast to 3.67 frames/second. However, Yahoo Detector significantly improves the detection accuracy, presenting 45.68 % less FP for Pornography-2k and 21.64 % less for UCF101 when compared to NuDetective. In contrast, regarding the FN rates, NuDetective

outperforms Yahoo Detector by 2.47 %. This is because NuDetective performs detection based on skin-tone. As a result, the detection throughput is significantly improved, and the FP rates are also significantly higher compared to a CNN-based approach.

Table 1: Accuracy and throughput of pornography detection approaches

Detection Technique	Detection Throughput (Frames/sec)	Dataset		
		Pornography-2k		UCF101
		FP (%)	FN (%)	FP (%)
Yahoo Detector [6]	3.67	12.80	5.53	1.61
NuDetective [30]	411.64	58.48	3.06	23.25

4.3 Discussion

Using the evaluated techniques for the detection of pornographic content in videos, it has been shown that they are unable to cope with real-time detection, even when executed using dedicated hardware. These techniques present either a low throughput or a significantly high error rate. Consequently, they cannot be executed in resource-constrained devices, which are environments where pornography content is often shared and viewed, either owing to their low accuracy or high computational demands. Nonetheless, the obtained accuracy rates render these detection techniques as unreliable. Therefore, owing to a high FP rate, the user no longer trusts the detection approach, recalling that the best FP rate is obtained using Yahoo Detector with 1.61 % for the UCF101 dataset. In addition, given that these techniques classify evaluated video as either pornography or not, in the case of FP, the user is not able to view the entire video. Therefore, it is imperative that pornography detection techniques can be implemented on resource-constrained devices, perform real-time detection, and appropriately address the challenge of FP in a user-friendly manner. This will facilitate reliable and appropriate use in production environments.

5 PRIVATE PARTS CENSOR

To address the real-time detection requirement and lack of reliability in the classification of pornographic content in videos, we present PPCensor. The goal is to achieve real-time and user-friendly detection without the need for additional processing on the user’s device. To achieve this objective, PPCensor is executed on dedicated, but general-purpose hardware and detection is performed transparently on the end-user device. The operation of PPCensor proceeds in two main stages: *Video-Streaming Query and Handling*, and *Pornography Object Detection*.

The *Video-Streaming Query and Handling* addresses user device query parsing, video download, video filtering, and related filtered response. The process is entirely executed in a proxy server. Therefore, the only requirement is that the end-user should change the device settings to redirect queries to the PPCensor server. Therefore, the detection process does not require additional device processing, and it can be readily executed even in resource-constrained devices.

The *Pornography Object Detection* stage is responsible for presenting a user-friendly detection approach. To this end, PPCensor is the first work that addresses the task of pornography detection as an object recognition problem. This is based on the insight that private parts (explicit nudity) can be identified as an object and properly filtered in videos, e.g., blurring/obfuscating video region/frames. As a result, even if misclassification occurs, the user experience is not significantly degraded. This is because PPCensor treats inappropriate content as an object recognition problem and is able to filter inappropriate video frames instead of blocking the entire video content.

The PPCensor processing architecture is illustrated in Figure 1. These two stages are described in detail in the next subsections, including the architecture components, and the object recognition applied to the detection of pornographic video.

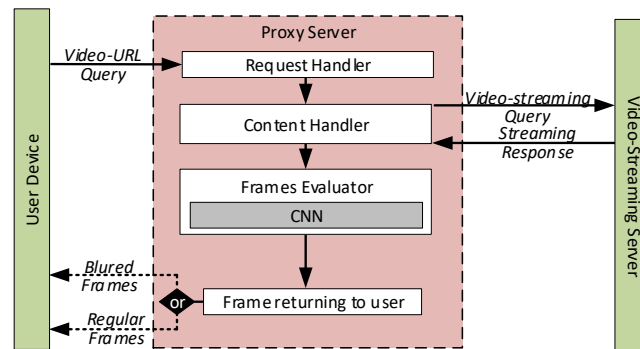


Figure 1: PPCensor processing architecture.

5.1 Pornography Object Detection

Before examining the details of the PPCensor processing architecture, we first describe its novel detection approach. In contrast to related works that address the detection of pornographic video via the detection of entire video frames, PPCensor addresses pornographic content detection as an object recognition problem. Therefore, PPCensor is able to provide two important benefits: detection throughput, and user-friendly detection. First, the detection of image objects can be achieved in near real-time. This is because several CNN architectures are used to address object recognition tasks as a regression problem, significantly increasing the detection throughput. Second, the filtering of specific image regions (objects), instead of the entire image significantly improves the user experience because if an FP occurs, only a part of the video image will be blurred as opposed to blocking the entire video.

To achieve this objective, the PPCensor considers each private part, both female and male, as an object for recognition. As such, the objective of the PPCensor is to recognize four specific object classes, we considered the following as private parts (PP): *male and female sexual organs (penis and vagina)*, *female breasts*, and *buttocks*. It is important to note that these private parts are not commonly shared in normal videos; in contrast, they are often present in pornographic videos. For instance, in a single frame, an explicitly pornographic movie may show all private parts, depending only on the camera position.

Therefore, by considering private parts as objects for recognition, PPCensor is able to detect pornographic videos using CNN-based object recognition techniques.

5.2 Processing Architecture

The objective of PPCensor is to perform object recognition based on pornographic content in real-time without any additional processing on user devices. In this regard, PPCensor is implemented as a proxy server, and the processing architecture is illustrated in Figure 1.

The architecture considers a set of *user devices* that need to assess and filter their video media. To this end, the user should only configure their device to forward any queries to the PPCensor proxy server, thus, using it as a network proxy. Therefore, all video-streaming queries performed on the device are forwarded to the PPCensor. The PPCensor server, in turn, receives the performed queries via the *Request Handler* module, which is responsible for URL (Uniform Resource Locator) query receiving and parsing. As such, the goal of the module is to establish a video query URL for further processing.

Subsequently, the *Content Handler* module queries the video-streaming server for the video frames. Consequently, it downloads the user device queried streaming frames to the proxy server and forwards it to the next module, the *Content Evaluator*. The module, in turn, executes the proposed *pornography Object Detection* engine and appropriately labels all identified image objects (described in Section 5.1). Finally, the *Content Filter* module performs filtering of the frames received by the user device in real-time, e.g., blurring the identified pornographic objects or even blocking the video data.

5.3 Discussion

Different from the literature, which does not consider detection scheme demanded processing, our proposed architecture for real-time detection of pornographic content in videos is based on two main insights. First, our work is the first approach that addresses pornographic content detection as an object recognition problem. Therefore, PPCensor is able to improve the user experience, considering that an FP does not block the user’s access to the queried video, but rather, only blurs specific frame regions. Second, we provide real-time pornographic content detection without any additional processing of the user device. This is because PPCensor is implemented as a network proxy. Consequently, the accessed video is evaluated in a transparent and remote manner, without the need for user interaction.

6 PROTOTYPE

A PPCensor prototype was implemented and deployed in a multithreaded process as shown in Figure 2. The prototype considers three main entities: *User Device*, *Proxy Server* (implementation of PPCensor), and *Video-Streaming Server*.

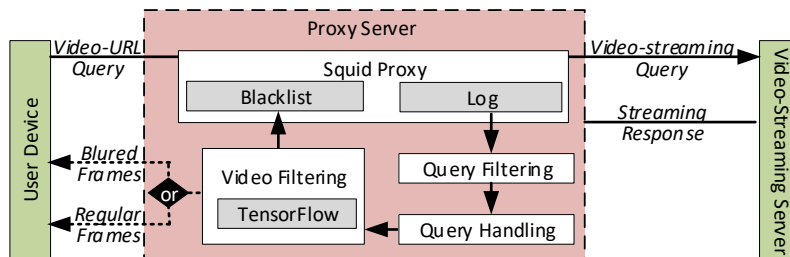


Figure 2: PPCensor prototype implementation architecture.

The *Video-URL Query* process is responsible for video querying in PPCensor. In production deployment settings, a *Video-URL Query* is represented by the user device request. To this end, a Python program was implemented to perform several *Video-URL* queries. All the performed queries were passed to the *PPCensor* (Proxy) server that was implemented using Squid Proxy, a web supporting proxy server application. The *Video-URL Query* node was configured to use *PPCensor* as its proxy server. As a result, there were no additional changes to the end-user device.

The *PPCensor* server was implemented using three main processes. First, proxy implementation was executed using Squid Proxy [46] version 4.5. This tool is publicly available and widely used for network proxy in Linux-based servers. Over time, the Squid Proxy logs all received queries in a log file, which is then parsed and interpreted by another process, the *Query Filtering and Handling*. These processes are modules that are implemented in Python, and continuously filter the Squid Proxy log. When a video-URL is found, the corresponding media URL is downloaded from the *Video-Streaming Server* via the Pafy API [47] version 0.5.1. The process performs video download in 0.5 frames/second. As such, a single frame is downloaded for every 2-second video interval, which is temporarily stored (cached) in a folder for further processing. Finally, the proposed *Pornography Object Detection* approach was implemented using Python in another process. The process executes the TensorFlow Detect Object API [48] using tensorflow-gpu version 1.14. Thus, the module is executed continuously, and when a new video frame is downloaded, the CNN model is applied, and the identified pornography objects are labeled. The module also enables video blacklisting via the Squid Proxy blacklist feature. Therefore, a pornography classified video may also be blocked in real-time if needed.

Finally, the *Video-streaming Server* is responsible for answering both the *video-URL Query* and the *PPCensor video-streaming* queries. It is important to note that if the *PPCensor* server blacklist finds a pornography URL, the blacklisted URL download is promptly interrupted. As such, the end-user is not able to further access the video even if video streaming is currently under download.

7 EVALUATION

We present the PPCensor analysis by answering four research questions. R1. Is it possible to treat the detection of pornographic video as an object recognition task? R2. How challenging is the detection of each private part? R3. How does PPCensor perform for image classification tasks compared to state-of-the-art CNNs? R4. Does PPCensor facilitate real-time detection of pornographic streaming video?

The next subsections outline the details of the developed dataset used in our work, and the PPCensor evaluation in terms of both accuracy and throughput.

7.1 A Private Parts Object Dataset

In general, in the literature, the provided data is labeled for the evaluation of pornographic image-based or video-based detection schemes, rather than object-based approaches. Consequently, to evaluate the proposed PPCensor detection engine, we built a novel dataset called *Private Parts Object Dataset* (PPO).

The PPO dataset was built based on the well-known Pornography-2k [34] dataset. However, given that we tackle pornography detection as an object recognition problem, we preprocessed 1000 pornographic videos. The preprocessing task filters video frames according to their content. As such, only explicitly pornographic frames are selected. From this subset, we manually selected object regions as either *penis*, *vagina*, *female breasts*, or *buttocks* in each video frame. To achieve this goal, we set a bounding box region that contains each considered pornographic object. As a result, the final dataset consisted of 50,870 pornographic frames from which we selected 52,215 objects containing 13,607 *penis*, 11,346 *vaginas*, 13,963 *female breasts*, and 13,299 *buttocks*. Note that a single video frame may contain more than one target object. For the model building procedure, the dataset was split into three parts: *training*, *validation*, and *test*; each dataset part was made of 60 %, 20 %, and 20 % of the built dataset videos respectively. Consequently, each dataset part is made of unique videos, each with unique pornographic objects, thereby properly evaluating the generalization capacity of the method. The *training* dataset is used for learning purposes, the *validation* dataset for the model training adjustment process, and the *test* dataset for final model evaluation.

7.2 The Accuracy Performance of PPCensor

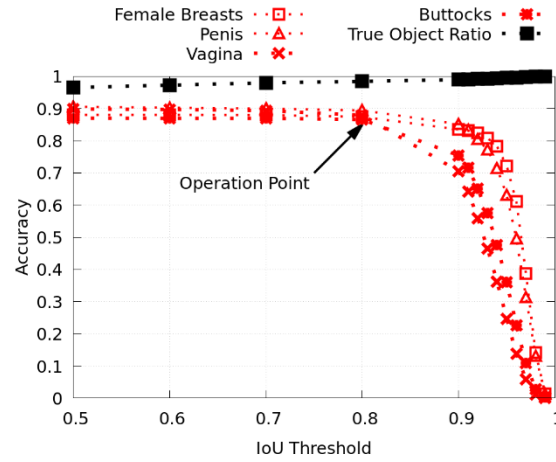
To evaluate PPCensor object-based detection feasibility, we first address R1. To achieve this objective, we evaluate several object recognition CNN-based approaches in the PPO dataset. The evaluated techniques were executed in a desktop equipped with an Intel I7 CPU (quad-core), 16GB of memory, and a Nvidia Titan-XP GPU.

Each CNN architecture was executed for 400,000 steps. The learning rate and decay rate were empirically set for each architecture after several essays. In addition, all evaluated CNN architectures were trained using transfer learning [20-21]. Thus, the CNNs used the obtained weights in the COCO dataset [28], a widely used dataset for object recognition purposes. The CNN-based techniques were evaluated with respect to their detection throughput and mAP. The mAP was computed according to an intersection over union (IoU), applying a threshold of 0.5 for each object bounding box. Table 2 shows the performance of the CNN-based object recognition architecture.

Table 2: PPCensor performance evaluation of pornography object detection

CNN Architecture	Frames/sec	mAP (IoU 0.5) %
Faster R-CNN Inception v2	10.87	63.50
Faster R-CNN NAS	1.19	56.53
Faster R-CNN ResNet 50	8.33	62.55
SSD MobileNet	59.55	55.92

It can be observed that there are significant differences among the accuracy values obtained for each evaluated CNN architecture. Regarding the most accurate CNN, Faster R-CNN Inception, a mAP of 63.50 % was achieved. It is important to note that the mAP considers the entire identified object intersection of at least 50 %, in this evaluation. However, in terms of production environment usage, the identified bounding box region may be increased (i.e., increasing the blurred region) to improve the model’s accuracy without significant user experience tradeoff. Consequently, the achieved accuracy for the most accurate architecture of 63.50% of mAP for the Faster R-CNN Inception, shows that it is feasible to treat pornographic video-streaming as an object recognition task.

**Figure 3: Faster R-CNN Inception IoU and accuracy tradeoff.**

We have further investigated the relation between IoU threshold and accuracy rates, as shown in Figure 3. It can be observed that the IoU threshold is able to improve the true object ratio with a small tradeoff for each object detection rate up to 0.8 IoU threshold. As the accuracy rates remain similar up to an IoU threshold of 0.8, whereas the true object rate improves slightly from 3.84% to 1.53%.

Table 3: Faster R-CNN Inception (IoU 0.8) architecture confusion matrix.

Object Class	Classified as				
	<i>False-Negative (FN)</i>	<i>Female Breasts</i>	<i>Penis</i>	<i>Vagina</i>	<i>Buttocks</i>
<i>Female Breasts</i>	271	2186	38	0	5
<i>Penis</i>	247	2	2236	11	4
<i>Vagina</i>	258	0	4	2199	39
<i>Buttocks</i>	285	5	7	38	2165

Finally, to address question R2, we further inspect the detection rate of each private part. To achieve this objective, we first evaluate the Faster R-CNN Inception with an IoU of 0.8 (Operation Point, Figure 3), and a confusion matrix as shown in Table 3. It can be observed that in general, private parts are not misclassified, but rather, not identified as an object (Table 3, FN). This means that CNN is able to identify which type of private part is found. In addition, two objects present a higher FP occurrence, the female breasts, and penis.

Figure 4 shows examples of pornographic images labeled with Faster R-CNN Inception architecture. In the figure, it is possible to note how the proposed object recognition approach impacts the user experience. The regions in the images with private part objects are properly identified and filtered, and the user experience is not significantly degraded. This is because the user is still able to identify the scene context without being exposed to explicit nudity.

Figure 5 shows examples of non-pornographic images with misrecognized objects (FP – False Positive) when the Faster R-CNN Inception architecture is applied. Common examples of FP include an arm or finger misidentified as *Penis* (Figure 5-c, 5-d, 5h), and rounded regions being mislabeled as *Female Breasts* (Figure 5-a, 5-e, 5-g). In the figure, it is possible to note that a misidentification

of an object does not introduce a significant impact on user experience. This means that the end-user experience is not significantly degraded when an FP occurs. This is because only a small portion of the video is filtered or blurred.

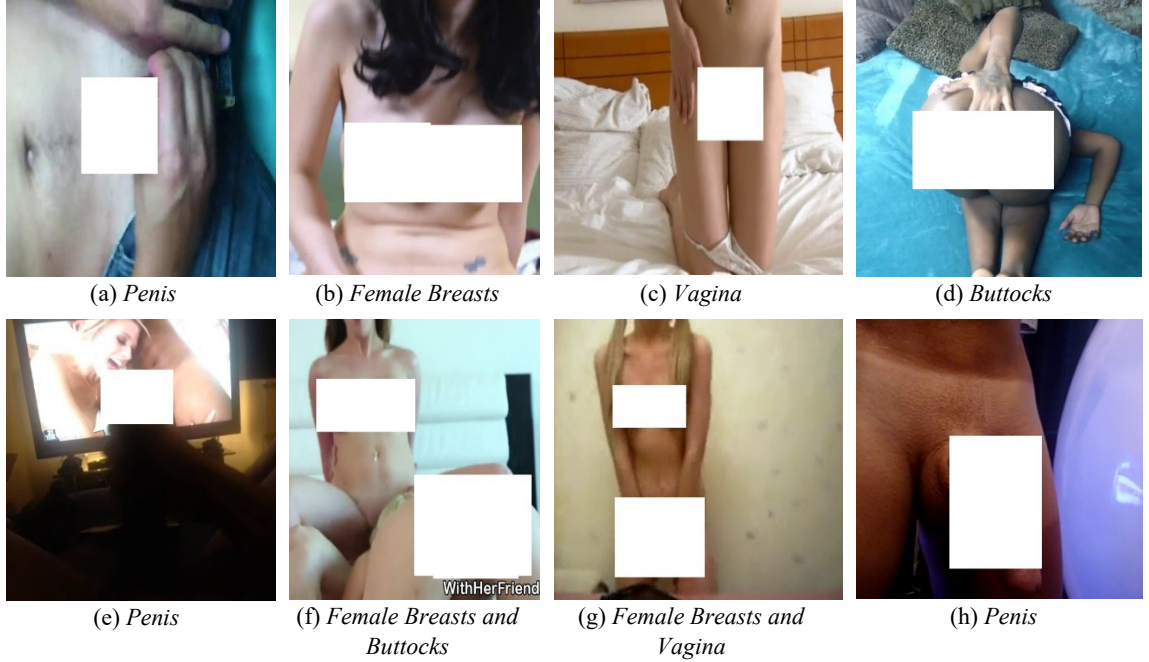


Figure 4: PPCensor with Faster R-CNN Inception (IoU 0.8) identification examples of Private Parts.

Finally, in some circumstances, the system administrator may prefer to block the entire video, or portions of it, instead of filtering specific video frame regions as performed by PPCensor. For instance, in the case of explicitly pornographic movies, it is preferable to block access to the entire movie instead of allowing access to a filtered video. Hence, to address question R3, we have also evaluated our proposed approach during application to the image classification field. To achieve this objective, PPCensor was customized to classify a video frame as pornographic if any private objects were identified in the video frame. The PPCensor results were also compared to state-of-the-art CNN architectures used for image classification tasks.



Figure 5: UCF101 dataset common FP examples when using the Faster R-CNN Inception (IoU 0.8).

In this case, the Pornography-2k dataset was split into training, testing, and validation datasets comprising 60 %, 20 %, and 20 % of the videos respectively. The CNNs were trained using transfer learning [20-21] based on the training dataset, weight adjustment was performed using the validation dataset, and finally, testing was conducted using the test dataset. Each CNN (CaffeNet [49], AlexNet [22], and Inception [19]) architecture was executed for 400,000 steps. The learning rate and decay rate were individually empirically set for each architecture after many essays. The obtained models were also evaluated in the UCF101 dataset, in this case, any video frame classified as pornography was assumed to be FP, considering that the dataset consisted of only normal video frames.

Table 4: PPCensor comparison when applied to image classification tasks.

Detection Technique	Detection Throughput (Frames/sec)	Dataset		
		Pornography-2k (test dataset)		UCF101
		FP (%)	FN (%)	FP (%)
Yahoo Detector [6]	3.67	12.80	5.53	1.61
NuDetective [30]	411.64	58.48	3.06	23.25
CaffeNet [49]	63.97	3.41	3.70	9.68
Alexnet [22]	61.12	5.08	4.61	13.69
Inception [19]	18.29	2.46	2.03	8.03
PPCensor (Faster R-CNN Inception)	10.87	1.66	2.34	3.41

Table 4 shows the PPCensor performance in comparison with traditional image recognition techniques for Pornography-2k and UCF101 datasets. Surprisingly, PPCensor outperforms the FP rates of all evaluated state-of-the-art techniques in the Pornography-2k dataset, while also outperforming all but Yahoo Detector in the UCF101 dataset. Regarding the FN rates, PPCensor also presents similar results to the other techniques, achieving 2.34 % for the Pornography-2k dataset, an increase of only 0.31 % of the FN rate for the best CNN architecture. However, the PPCensor detection throughput is significantly worse compared to the other evaluated CNN architectures. Therefore, one could improve our proposed technique throughput by applying the other evaluated object detection CNN architectures (Table 2). For instance, a rate of 1.19 frames/second was obtained with only a 56.53 mAP tradeoff. In addition, frame evaluation is only a part of the PPCensor classification pipeline (Figure 2). Therefore, despite the application of a CNN with a high classification throughput, the classification pipeline must also be improved. Specifically, real-time video downloads and filtering should also be addressed. Hence, in the next subsection, the PPCensor classification pipeline throughput and connection scalability for real-time video classification tasks are further evaluated.

7.3 PPCensor Performance and Scalability

Finally, to answer address R4, we evaluate whether PPCensor facilitates transparent and real-time detection of pornographic content without any additional processing of the monitored device using a proxy server. To this end, two nodes were used; the user device and the PPCensor. Both nodes were equipped with an Intel I7 CPU (quad-core), and 16GB of memory. In addition, the PPCensor node was also equipped with a Nvidia Titan-XP GPU, which executes the Faster R-CNN Inception architecture for object recognition tasks (implementation details are described in Section 6), with a mAP threshold of 0.8.

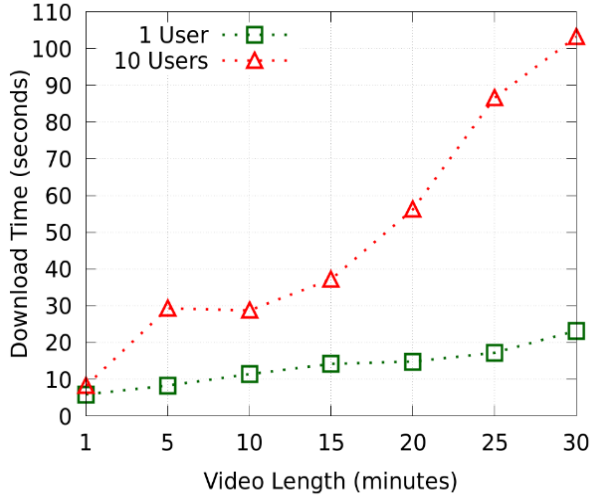
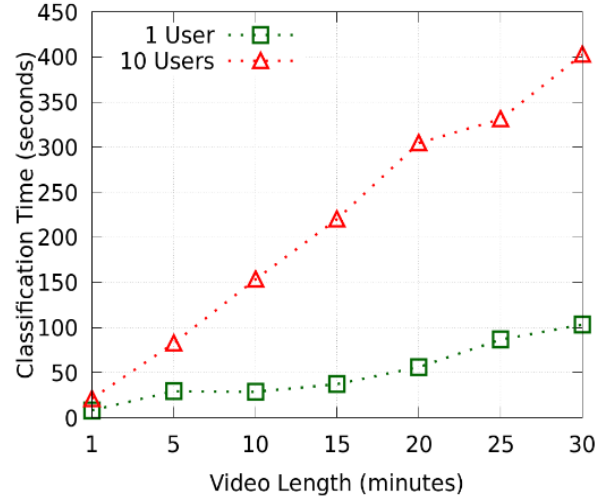
**Figure 6: PPCensor video download time.****Figure 7: PPCensor video classification time.**

Figure 6 shows the tradeoff between video length and download time according to the number of connections. It is observed that the PPCensor is able to download video for up to 10 user connections in real-time, considering that the download time is always less than the video length. Moreover, download time increases linearly for a 1-user scenario, whereas there is a significant increase when executing for 10 users. This is because the video download is both network-bound and CPU-bound. Therefore, given that the evaluated node contains only 8 threads, the execution time significantly degrades after a specific video length threshold (20 minutes).

Figure 7 shows the tradeoff between video length and classification time. It can be observed that the execution time increases almost linearly for both 1-user and 10-user scenarios. This behavior is mainly because the classification task is GPU-bound. Therefore,

the evaluation of video frames is performed individually, rather than simultaneously. Only one model is loaded in memory for the implementation of the prototype.

Figure 8 shows the tradeoff between time and video length for the entire PPCensor process for both video download and frame classification. Similarly, the detection can be performed in real-time for both 1-user and 10-user scenarios. This is because the process execution time remains lower than the video length. In addition, the required processing time does not significantly increase compared to the classification time (Figure 7). This occurs due to the GPU-bound nature of the classification task and the CPU-bound nature of the download process. Therefore, each process, classification, and download, does not degrade each other. As a result, PPCensor enables the detection of pornographic video in near real-time without any additional processing on the user device.

Finally, Figure 9 evaluates the relation between the number of users and the execution time for 5-minute long videos. In this case, PPCensor can accommodate up to 35 users watching videos in near real-time with standard quality, and 25 users watching high-quality videos. As such, PPCensor transparently detects pornographic video streaming in real-time for up to 35 users without incurring any additional device processing in a user-friendly manner. It is important to note that the prototype implementation could be fine-tuned to improve several implementation details, thereby further increasing the detection throughput.

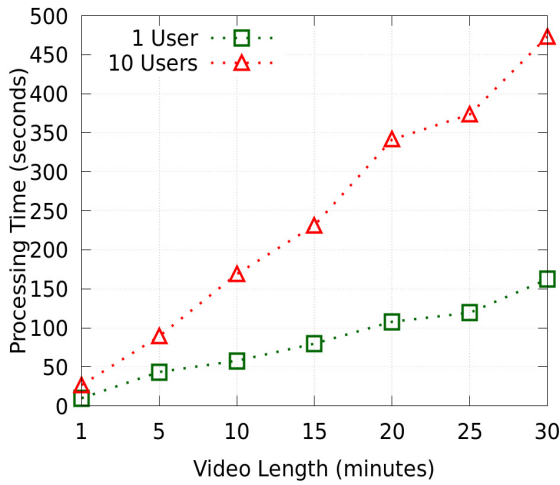


Figure 8: PPCensor video download and classification time.

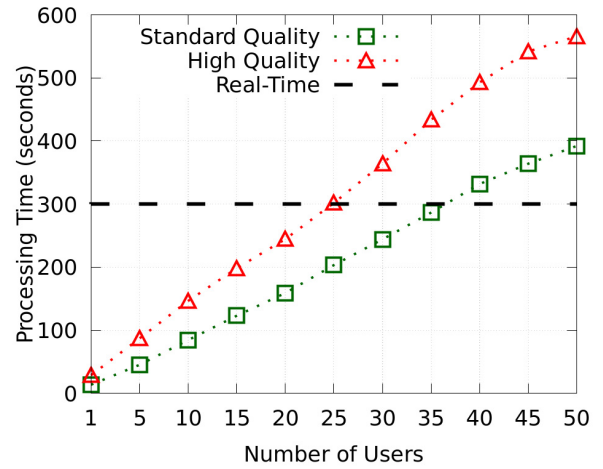


Figure 9: PPCensor processing time for 5-minute long standard and high-quality videos. The proposed approach is able to handle 25 users viewing standard quality video and 35 users viewing high-quality videos in real-time.

8 CONCLUSION

Current publicly available tools for the detection of pornographic content in video streaming do not facilitate transparent user-friendly detection. In this paper, we have proposed PPCensor, which is based on two main insights with respect to pornographic content detection. First, PPCensor is the first work that treats pornography content as an object recognition problem. Consequently, it provides user-friendly detection with no significant impact when FP occurs. The evaluation based on the manual analysis of over 50 thousand frames of private parts revealed that the proposed technique can detect inappropriate content in near real-time. Moreover, it generates results that are similar to cute-edge proposals for image classification tasks. Second, PPCensor architecture is implemented as a proxy server; therefore, it does not incur additional processing in the user device while performing transparent execution. The evaluation performed for a desktop with a typical configuration shows that PPCensor is able to accommodate up to 35 simultaneous user connections in real-time. The PPO dataset, as well as the PPCensor source code, can be provided upon request to the reader.

ACKNOWLEDGMENTS

The authors thank CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico) for partial financial support for the project, process 430972/2018-0, and NVIDIA Corporation for donating a Titan-XP GPU used in the experiments. Jackson Mallmann would like to thank the IFC (Federal Institute Catarinense) and CAPES (Coordenação de Aperfeiçoamento de Pessoal de Nível Superior) for the scholarship granted through public notice N°. 231/2017.

REFERENCES

- [1] Statista. Available online: <https://www.statista.com/chart/16959/share-of-the-internet-that-is-porn/>. (Accessed: 17 October 2019).
- [2] Pornhub. Available online: <https://www.pornhub.com/insights/2018-year-in-review/>. (Accessed: 17 October 2019).
- [3] Prevent Child Abuse America. Available online: <https://preventchildabuse.org/>. (Accessed: 17 October 2019).
- [4] Enough, H., Pornography Statistics. Available online: http://enough.org/stats_porn_industry/. (Accessed: 17 October 2019).

- [5] M. Moustafa. Applying deep learning to classify pornographic images and videos. 7th Pacific-Rim Symposium on Image and Video Technology (PSIVT) (2015).
- [6] J. Mahadeokar and G. Pesavento. 2016. Open sourcing a deep learning solution for detecting nsfw images. Available online: <https://yahooeng.tumblr.com/post/151148689421/open-sourcing-a-deep-learning-solution-for-> (Accessed: 17 October 2019).
- [7] M. Perez, S. Avila, D. Moreira, D. Moraes, V. Testoni, E. Valle, S. Goldenstein and A. Rocha. 2017. Video pornography detection through deep learning techniques and motion information. In *Neurocomputing*, Volume 230, 2017, Pages 279-293, ISSN 0925-2312, <https://doi.org/10.1016/j.neucom.2016.12.017>
- [8] J. Wehrmann, G. S. Simões, R. C. Barros, and V. F. Cavalcante. 2018. Adult content detection in videos with convolutional and recurrent neural networks. *Neurocomputing*. 272, C (January 2018), 432-438. DOI: <https://doi.org/10.1016/j.neucom.2017.07.012>
- [9] X. Ou, H. Ling, H. Yu, P. Li, F. Zou, and S. Liu. 2017. Adult Image and Video Recognition by a Deep Multicontext Network and Fine-to-Coarse Strategy. *ACM Trans. Intell. Syst. Technol.* 8, 5, Article 68 (July 2017), 25 pages. DOI: <https://doi.org/10.1145/3057733>
- [10] F. Idris and S. Panchanathan. 1997. Review of image and video indexing techniques. *Journal of visual communication and image representation*, 8(2), 146-166.
- [11] Y. Kuroki, T. Nishi, S. Kobayashi, H. Oyaizu, and S. Yoshimura. 2007. A psychophysical study of improvements in motion-image quality by using high frame rates. *Journal of the Society for Information Display*. 15, 1 (2007), 61. DOI: <https://doi.org/10.1889/1.2451560>
- [12] The state of digital in April 2019: all the numbers you need to know. Available online: <https://wearesocial.com/blog/2019/04/the-state-of-digital-in-april-2019-all-the-numbers-you-need-to-know>. (Accessed: 17 October 2019).
- [13] W. Rawat and Z. Wang. 2017. Deep Convolutional Neural Networks for Image Classification: A Comprehensive Review. *Neural Computing*. 29, 9 (September 2017), 2352-2449. DOI: https://doi.org/10.1162/neco_a_00990
- [14] S. Pouyanfar, S. Sadiq, Y. Yan, H. Tian, Y. Tao, M. P. Reyes, M. Shyu, S. Chen, and S. S. Iyengar. 2019. A Survey on Deep Learning: Algorithms, Techniques, and Applications. *ACM Computing Surveys (CSUR)*, v.51 n.5, p.1-36, January 2019. doi:10.1145/3234150
- [15] M. Imani, D. Peroni, Y. Kim, A. Rahimi, and T. Rosing. 2017. Efficient neural network acceleration on GPGPU using content addressable memory. *Proceedings of the 2017 Design, Automation and Test in Europe, DATE 2017*. (2017), 1026-1031. DOI: <https://doi.org/10.23919/DATE.2017.7927141>
- [16] H. Park, D. Kim, J. Ahn, and S. Yoo. 2016. Zero and data reuse-aware fast convolution for deep neural networks on GPU. 2016 International Conference on Hardware/Software Codesign and System Synthesis, CODES+HSSS 2016. (2016), 1-10. DOI: <https://doi.org/10.1145/2968456.2968476>
- [17] A. Karpathy. 2016. CS231n: Convolutional Neural Networks for Visual Recognition. Available online: <http://cs231n.github.io/convolutional-networks/>. (Accessed: 17 October 2019).
- [18] K. He, X. Zhang, S. Ren, and J. Sun. 2015. Deep residual learning for image recognition. In *Proc. CoRR*, 2015.
- [19] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna. (2016). Rethinking the inception architecture for computer vision. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition*, pages 2818-2826, 2016.
- [20] M. Oquab, L. Bottou, I. Laptev, and J. Sivic. Learning and transferring mid-level image representations using convolutional neural networks. In *Proc. IEEE conf. on computer vision and pattern recognition*, 2014, p. 17171724.
- [21] C. Tan, F. Sun, T. Kong, W. Zhang, C. Yang, and C. Liu. 2018. A Survey on Deep Transfer Learning. <http://arxiv.org/abs/1808.01974>
- [22] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. *Communications of the ACM*. 60, 6 (May 2017), 84-90. DOI: <https://doi.org/10.1145/3065386>
- [23] Z. Zhao, P. Zheng, S. Xu, X. Wu. (2019). Object Detection with Deep Learning: A Review. *IEEE Transactions on Neural Networks and Learning Systems*. PP. 1-21. 10.1109/TNNLS.2018.2876865. DOI: 10.1109/TNNLS.2018.2876865
- [24] J. Huang, V. Rathod, C. Sun, M. Zhu, A. Korattikara, A. Fathi, I. Fischer, Z. Wojna, Y. Song, S. Guadarrama, and K. Murphy. 2017. Speed/accuracy trade-offs for modern convolutional object detectors. *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.
- [25] S. Ren, K. He, R. Girshick, and J. Sun. 2017. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137-1149, 2017.
- [26] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C. Fu, A. C. Berg. 2016. SSD: Single Shot MultiBox Detector. *Computer Vision -- ECCV 2016*, Springer International Publishing.
- [27] J. Redmon, A. Farhadi, Yolov3: An incremental improvement, *CoRR*, 2018.
- [28] T. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C.L. Zitnick. 2014. Microsoft COCO: Common objects in context. In *ECCV*, 2014.
- [29] S. Karamizadeh and A. Arabsorkhi. (2018). Methods of pornography Detection: Review. In *Proceedings of the 10th International Conference on Computer Modeling and Simulation (ICCMS 2018)*. ACM, New York, NY, USA, 33-38. DOI: <https://doi.org/10.1145/3177457.3177484>
- [30] M. Polastri and P. Eleuterio. 2010. Nudetective: a forensic tool to help combat child pornography through automatic nudity detection. In *Proc. Workshop on Database and Expert Systems Applications (DEXA)*, 2010.
- [31] R. Ap-apid. An Algorithm for Nudity Detection. *Proceedings of the 5th Philippine Computing Science Congress*, 2005.
- [32] F. Nian, T. Li, Y. Wang, M. Xu, J. Wu. Pornographic image detection utilizing deep convolutional neural networks. *Neurocomputing*, 210 (2016), pp. 283-293. DOI: <https://doi.org/10.1016/j.neucom.2015.09.135>
- [33] R. Gordon. A calculated look at fixed-point arithmetic. *Embed. Syst. Progr.*, 11 (4) (1998), pp. 72-79.
- [34] D. Moreira, S. Avila, M. Perez, D. Moraes, V. Testoni, E. Valle, S. Goldenstein and A. Rocha. 2016. Pornography classification: The hidden clues in video space-time, *Forensic Science International*. Vol. 268, p. 46-61, 2016, DOI: 10.1016/j.forsciint.2016.09.010
- [35] S. Avila, N. Thome, M. Cord, E. Valle, and A.A. Araújo. 2013. Pooling in image representation: the visual codeword point of view. *Comput. Vision Image Understanding*, 117 (5) (2013), pp. 453-465.
- [36] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. 2015. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1-9, 2015.
- [37] P. Vitorino, M. Avila, S. Perez, and A. Rocha. Leveraging deep neural networks to fight child pornography in the age of social media. In *Proc. Journal of Visual Communication and Image Representation*, 2018.
- [38] L. Wang, J. Liao, and C. Xu. 2019. Vehicle Detection Based on Drone Images with the Improved Faster R-CNN. In *Proceedings of the 2019 11th International Conference on Machine Learning and Computing (ICMLC '19)*. ACM, New York, NY, USA, 466-471. DOI: <https://doi.org/10.1145/3318299.3318383>
- [39] G. Plastiras, C. Kyrkou, and T. Theodoridis. 2018. Efficient ConvNet-based Object Detection for Unmanned Aerial Vehicles by Selective Tile Processing. In *Proceedings of the 12th International Conference on Distributed Smart Cameras (ICDSC '18)*. ACM, New York, NY, USA, Article 3, 6 pages. DOI: <https://doi.org/10.1145/3243394.3243692>
- [40] B. Benjdira, T. Khursheed, A. Koubaa, A. Ammar, and K. Ouni. Car detection using unmanned aerial vehicles: Comparison between faster R-CNN and YOLOv3. In *Proc. 1st Int. Conf. Unmanned Vehicle Syst.-Oman (UVS)*, Feb. 2019, pp. 1-6.
- [41] P. Maheshwari, D. Alex, S. Banerjee, S. Behera, and S. Panda. 2018. Top View Person Detection and Counting for Low Compute Embedded Platforms. In *Proceedings of the 2018 the 2nd International Conference on Video and Image Processing (ICVIP 2018)*. ACM, New York, NY, USA, 35-43. DOI: <https://doi.org/10.1145/3301506.3301548>
- [42] L. Zhang, Y. A. Zhang, Z. Zhang, J. Shen, and H. Wang. Real-Time Water Surface Object Detection Based on Improved Faster R-CNN. 2019. doi:10.3390/s19163523. <https://www.mdpi.com/1424-8220/19/16/3523>
- [43] S. Saikia, E. Fidalgo, E. Alegre, L. Fernández-Robles. (2017). Object Detection for Crime Scene Evidence Analysis Using Deep Learning. In: Battiatto S., Gallo G., Schettini R., Stanco F. (eds) *Image Analysis and Processing - ICIAP 2017*. ICIAP 2017. Lecture Notes in Computer Science, vol 10485. Springer, Cham.
- [44] R. Zhao, W. Ouyang, H. Li, and X. Wang. 2015. Saliency detection by multi-context deep learning. In *Proceedings of the 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR'15)*. 1265-1274. DOI: <https://doi.org/10.1109/CVPR.2015.7298731>
- [45] K. Soomro, A. R. Zamir, and M. Shah. 2012. UCF101: A dataset of 101 human actions classes from videos. In *In the Wild*, 2012.
- [46] Squid Development Projects. Available online: <http://devel.squid-cache.org/>. (Accessed: 17 October 2019).

- [47] Pafy. Available online: <https://pythonhosted.org/pafy/>. (Accessed: 17 October 2019).
- [48] Tensorflow Object Detection API. Available online: https://github.com/tensorflow/models/tree/master/research/object_detection/. (Accessed: 17 October 2019).
- [49] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. 2014. Caffe: Convolutional Architecture for Fast Feature Embedding. In Proceedings of the 22nd ACM international conference on Multimedia (MM '14). ACM, New York, NY, USA, 675-678. DOI: <https://doi.org/10.1145/2647868.2654889>